

Debugging & Optimization Answers

Selected 10 Questions from Assessment Tool 3

Q1. Debugging Selection Sort Code

Identify errors in comparison and swapping logic, and explain their root causes.

Answer:

Errors & Causes: The comparison condition `if arr[j] > arr[min_idx]` finds the maximum element rather than the minimum, which incorrectly sorts the array in descending order. Additionally, the swapping logic is executed for every iteration inside the inner loop, overriding the purpose of selection sort.

Corrected Code (Time Complexity: $O(n^2)$):

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1): # Optimized outer loop
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]: # Fix: Correct comparison
                min_idx = j
        if min_idx != i: # Fix: Swap only if needed (outside inner loop)
            arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

Q2. Debugging Linear Search Code

Identify errors in loop conditions and return statements, and explain their causes.

Answer:

Errors & Causes: The function immediately returns -1 if the first element doesn't match the key, terminating prematurely. The return `i` is incorrectly placed outside the scope of a successful match.

Corrected Code (Time Complexity: $O(n)$):

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i # Fix: Return index on match
    return -1 # Fix: Return -1 only after full loop completes
```

Q3. Debugging Closest Pair Code

Identify logical issues in distance calculation and comparison.

Answer:

Errors & Causes: `min_dist` is initialized to 0, which is the absolute minimum distance possible. The condition `dist > min_dist` then seeks the *farthest* pair rather than the closest.

Corrected Code (Time Complexity: $O(n^2)$):

```
import math
def closest_pair(points):
    min_dist = float('inf') # Fix: Initialize to infinity
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            dist = math.sqrt((points[i][0] - points[j][0])**2 +
                             (points[i][1] - points[j][1])**2)
            if dist < min_dist: # Fix: Seek smaller distance
                min_dist = dist
    return min_dist
```

Q4. Debugging Convex Hull Logic

Identify logical errors in orientation checking.

Answer:

Errors & Causes: The provided code's cross-product formula for orientation lacks proper grouping or sign evaluation, causing interior points to be falsely identified as boundary edges depending on how the returned value is processed.

Corrected Logic (Time Complexity: $O(1)$ per check):

```
def orientation(p, q, r):  
    # Returns 0 if collinear, > 0 if clockwise, < 0 if counterclockwise  
    val = (q[1] - p[1]) * (r[0] - q[0]) - (q[0] - p[0]) * (r[1] - q[1])  
    if val == 0: return 0  
    return 1 if val > 0 else -1
```

Q5. Debugging Binary Search Code

Identify errors in boundary updates.

Answer:

Errors & Causes: The loop condition `low < high` skips checking the final element when the search space narrows to a single index (where `low == high`). Overflows can also happen in languages with bounded integers for `(low + high) // 2`.

Corrected Code (Time Complexity: $O(\log n)$):

```
def binary_search(arr, key):  
    low, high = 0, len(arr) - 1  
    while low <= high: # Fix: Use <= to check the single element  
        mid = low + (high - low) // 2 # Fix: Prevents overflow  
        if arr[mid] == key:  
            return mid  
        elif arr[mid] < key:  
            low = mid + 1 # Target is in right half  
        else:  
            high = mid - 1 # Target is in left half  
    return -1
```

Q6. Debugging Bubble Sort Code

Identify logical errors in loop conditions and swapping mechanism.

Answer:

Errors & Causes: `if arr[j] < arr[j+1]` sorts in descending order. The swapping syntax is invalid Python (`arr[j], arr[j+1]...`). The inner loop unnecessarily processes already sorted elements at the end.

Corrected Code (Time Complexity: $O(n^2)$ worst, $O(n)$ best):

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False # Optimization flag
        for j in range(n - 1 - i): # Fix: Ignore sorted tail
            if arr[j] > arr[j+1]: # Fix: > for ascending order
                arr[j], arr[j+1] = arr[j+1], arr[j] # Fix syntax
                swapped = True
        if not swapped: break # Early termination
    return arr
```

Q7. Debugging Insertion Sort Code

Identify errors in shifting logic and loop conditions.

Answer:

Errors & Causes: The condition `j > 0` misses the 0th index. The loop updates `j = 1` instead of decrementing `j -= 1`, resulting in an infinite loop. The final placement `arr[j] = key` overwrites the wrong index.

Corrected Code (Time Complexity: $O(n^2)$):

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key: # Fix: j >= 0
            arr[j+1] = arr[j]
            j -= 1 # Fix: Decrement j
        arr[j+1] = key # Fix: Place key at right index
    return arr
```

Q8. Debugging String Matching Code

Identify logical errors in indexing and bounds.

Answer:

Errors & Causes: The outer loop `range(n - m)` stops one index short of the last possible alignment. The check `if j == m - 1` might improperly report a match if a mismatch happens on the last character.

Corrected Code (Time Complexity: $O((n-m+1)m)$):

```
def string_match(text, pattern):
    n, m = len(text), len(pattern)
    for i in range(n - m + 1): # Fix: + 1 for last alignment
        match = True
        for j in range(m):
            if text[i+j] != pattern[j]:
                match = False
                break
        if match: # Fix: Use flag for correct match evaluation
            print("Match at index", i)
```

Q9. Debugging Maximum Element Search Code

Identify errors in initialization and comparison logic.

Answer:

Errors & Causes: Initializing `max_val = 0` causes the function to return 0 for arrays with all negative numbers. The condition `arr[i] < max_val` incorrectly searches for the minimum value.

Corrected Code (Time Complexity: $O(n)$):

```
def find_max(arr):
    if not arr: return None
    max_val = arr[0] # Fix: Initialize to first element
    for i in range(1, len(arr)):
        if arr[i] > max_val: # Fix: Find maximum
            max_val = arr[i]
    return max_val
```

Q10. Debugging Pair Sum Code (Brute Force)

Identify logical errors in loop structure and return conditions.

Answer:

Errors & Causes: The syntax `arr[i] + arr[j] = target` attempts an assignment instead of a comparison (`==`). The inner loop starting at 0 causes redundant mirror checks and duplicate outputs.

Corrected Code (Time Complexity: $O(n^2)$):

```
def pair_sum(arr, target):
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)): # Fix: Avoid permutations
            if arr[i] + arr[j] == target: # Fix: Use ==
                return (i, j)
    return None
```